Sujet de l'examen Master GEODATA

30/10/2026

Chargement des modules

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy.stats as sc

from sklearn.metrics import r2_score
from sklearn import model_selection
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split

from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.svm import SVR
```

In [2]:
```
# Création de la fonction pour le calcul des estimateurs des rangs des indices d
def compSRank(Y, X, I):
    """
    I est un indice de {1,...,p}
    X est une matrice de N lignes, chaque ligne a pour valeur (X1, X2, ..., Xp)
    Y est une la sortie
    """
    # Ordre selon X
    IS = np.argsort(X[: , I],axis = 0);

    # Sorties Y réordonnées selon X
    YRank = np.copy(Y)
    YRank = Y[IS]

    # Sorties shiftées
    YShift = np.roll(YRank,-1)

    # Calcul de l'estimation de l'indice de Sobol
    Cov = np.cov(YRank, YShift)
    SRank = Cov[0, 1] / Cov[0, 0]
    return SRank

# Création de la fonction pour calculer l'erreur
def BootSRank(Y, X, I, B):
    """
    I est un indice de {1,...,p}
    X est une matrice de N lignes, chaque ligne a pour valeur (X1, X2, ..., Xp)
    Y est une la sortie
    B nombre d'iterations
    """
    err = []
```

```
    for b in range(B):
        sample = np.random.choice(a=len(Y),size=len(Y),replace=True)
        err.append(compSRank(Y[sample],X[sample,:],I))
    errstd = np.std(err)
    return errstd
```

Contexte

Le cas d'application est celui du tsunami de 1887 qui est survenu suite à un séisme en Mer de Ligure près de Nice et de Gênes. Ce cas est celui vu en cours. On s'intéresse à 4 paramètres qui décrivent la rupture lors du séisme. On aimerait quantifier l'impact de ces incertitudes sur l'estimation de la hauteur de l'eau à la cote induite par le tsunami noté *Y*. La liste des paramètres sont :

- $X_1 = Z$, la profondeur (m)
- $X_2 = D$, le déplacement moyen lors de la rupture sur le plan de la faille (m)
- $X_3 = W$, la largeur de la rupture (m)
- $X_4 = L$, la longueur de la ruptre (m)

Un code de calcul a été utilisé par le BRGM pour estimer les valeurs de *Y* en fonction de **X**. Un ensemble de 300 réalisations aléatoires a été généré. Pour chaque réalisation de **X**, la valeur de *Y* a été estimée.

Objectifs

L'objectif est de conduire la

- **Propagation d'incertitude** pour quantifier l'impact des incertitudes de **X** sur *Y*
- **Analyse de sensibilité** pour identifier quelle incertitude *X* influence le plus *Y* et quantifier son influence.

Chargement des données

CODE A FAIRE TOURNER pour charger les données **X** et *Y*.

In [3]:
```
## Entrees
df0 = pd.read_csv("https://raw.githubusercontent.com/anrhouses/TD_MLUQ/refs/head
df = df0[['Z','D','W','L']]
X = df.values
```

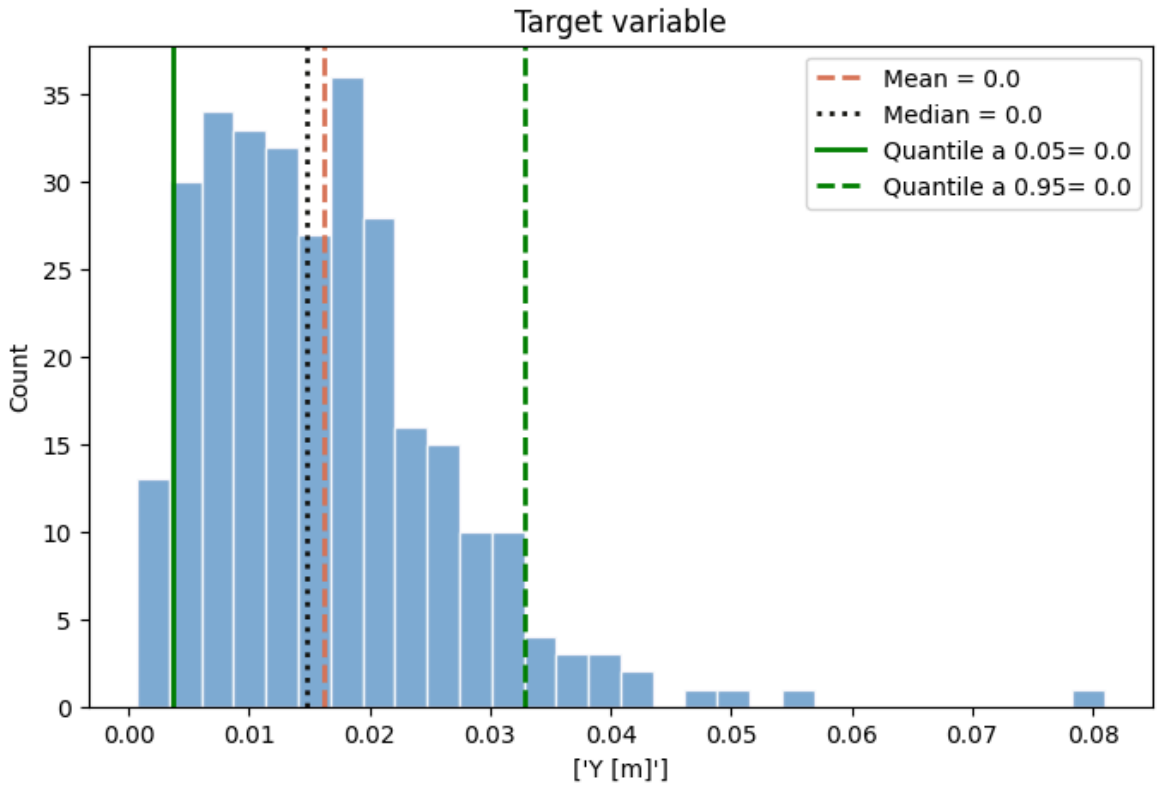
In [4]:
```
## Sortie
outputSample0  = pd.read_csv("https://raw.githubusercontent.com/anrhouses/TD_MLU
Y = outputSample0.values[:,0]
df['Y'] = Y
TARGET_LABEL = ["Y" + ' [m]']
```

Propagation d'incertitudes

QUESTION 1 Nous traçons l'histogramme de *Y* avec le code suivant ainsi que des caractéristiques statistiques. Que pouvez vous dire sur la distribution de *Y* ? Répondez dans les cases ci après. **Attention** pour écrire du texte assurer vous que la case est en mode *markdown*.

CODE DE LA QUESTION 1 A FAIRE TOURNER

In [5]:
```
fig, ax = plt.subplots(figsize=(8, 5))
ax.hist(Y, bins=30, edgecolor="white", alpha=0.85, color="#6a9bcc")
ax.axvline(np.mean(Y), color="#d97757", linestyle="--", linewidth=2, label=f"Mea
ax.axvline(np.median(Y), color="#141413", linestyle=":", linewidth=2, label=f"Me
ax.axvline(np.quantile(Y,0.05), color="green", linestyle="-", linewidth=2, label
ax.axvline(np.quantile(Y,0.95), color="green", linestyle="--", linewidth=2, labe
ax.set_xlabel(TARGET_LABEL); ax.set_ylabel("Count")
ax.set_title("Target variable")
ax.legend()
plt.show()
```

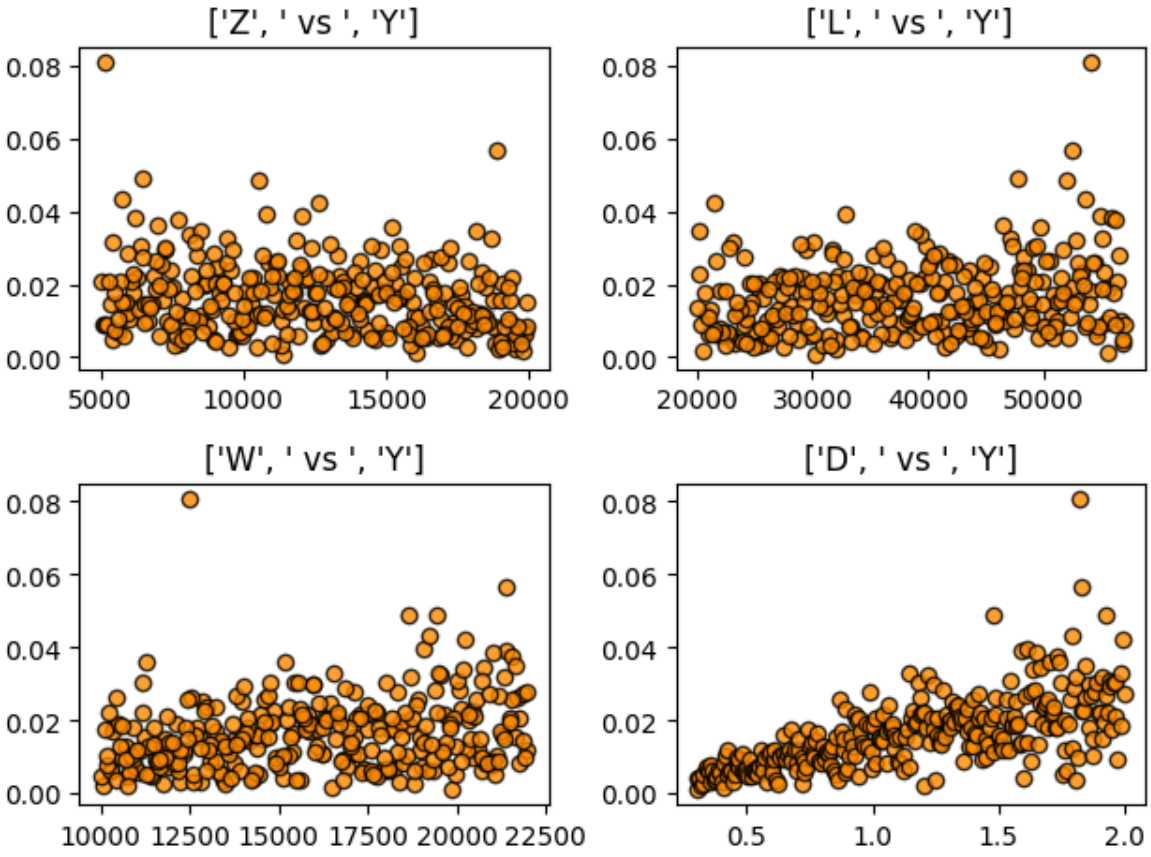


Analyse de sensibilité

QUESTION 2 Nous allons analyser la relation de chaque *X* avec *Y* en traçant les nuages de points avec le code suivant. Quelles hypothèses pouvez vous formuler sur ces relations ?

CODE DE LA QUESTION 2 A FAIRE TOURNER

In [6]:
```
fig, axs = plt.subplots(2,2)
axs[0,0].plot(df['Z'].values, Y, 'o', label='', markerfacecolor='darkorange', ma
axs[0,0].set_title(['Z'," vs ", "Y"])
axs[0,1].plot(df['L'].values, Y, 'o', label='', markerfacecolor='darkorange', ma
axs[0,1].set_title(['L'," vs ", "Y"])
axs[1,0].plot(df['W'].values, Y, 'o', label='', markerfacecolor='darkorange', ma
axs[1,0].set_title(['W'," vs ", "Y"])
axs[1,1].plot(df['D'].values, Y, 'o', label='', markerfacecolor='darkorange', ma
axs[1,1].set_title(['D'," vs ", "Y"])
fig.tight_layout()
```



QUESTION 3 Nous allons quantifier les corrélations entre chaque *X* et *Y* en utilisant le code ci-après. On utilise deux types de corrélation, celle linéaire et celle sur les rangs.

- Quelles informations sur les relations *X* - *Y* peut-on déduire de l'analyse des corrélations ?
- Quelles conclusions pouvez vous tirer de cette analyse ?
- Est-ce que votre analyse des nuages de points est en accord avec cette analyse de corrélation ?

CODE DE LA QUESTION 3 A FAIRE TOURNER

In [7]:
```
pred = df.columns.values.tolist()[:4]
correlation = df.iloc[:,0:5].corr().iloc[4,:4]
rank_correlation, rank_correlation_pval = sc.spearmanr(df.iloc[:,0:5])
rank_correlation = rank_correlation[4,:4]
dcorr = {
        "Z": [correlation.values[0], rank_correlation[0]],
```

```
        "D": [correlation.values[1], rank_correlation[1]],
        "W": [correlation.values[2], rank_correlation[2]],
        "L": [correlation.values[3], rank_correlation[3]]
       }
dcorr = pd.DataFrame(data=dcorr,
        index=['Correlation lineaire', 'Correlation sur les rangs'])
dcorr
```

Out[7]:

	Z	D	W	L
Correlation lineaire	-0.202788	0.657768	0.249591	0.206310
Correlation sur les rangs	-0.195948	0.713530	0.255237	0.177231

QUESTION 4 Nous allons conduire une analyse globale de sensibilité en calculcant les indices de Sobol' de premier order. Pouvez vous rappeler quelle interprétation on peut donner à ces indices ?

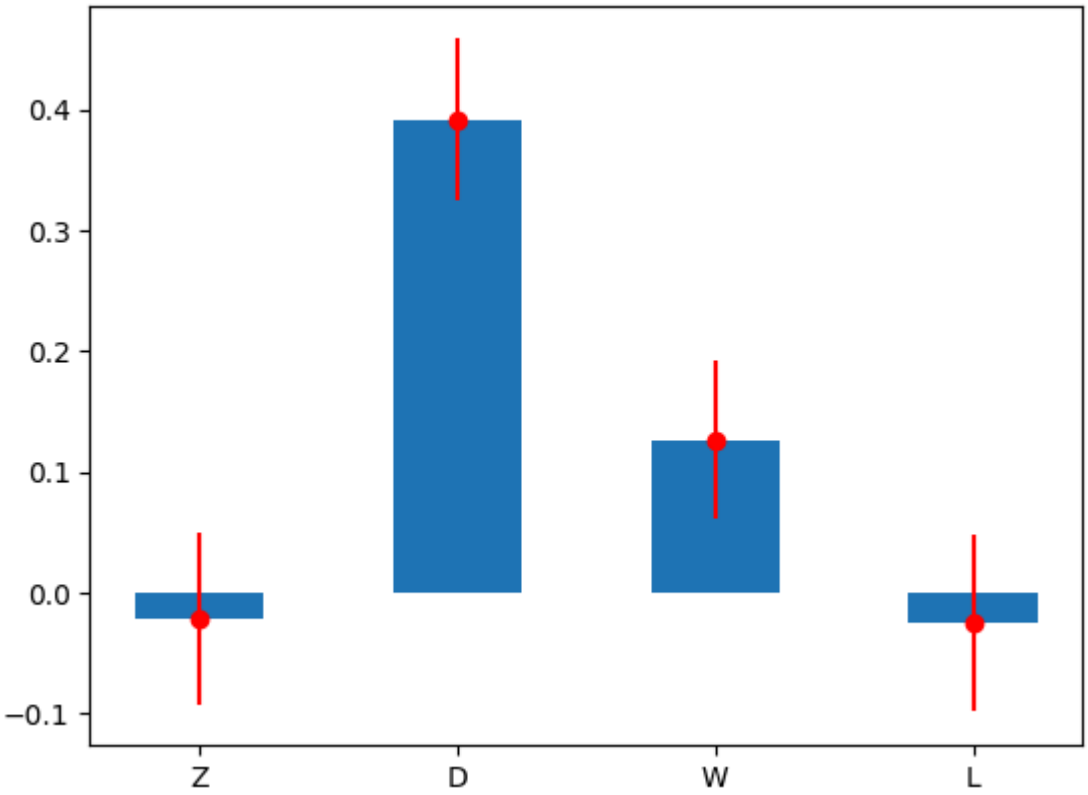
QUESTION 5 Nous allons les calculer avec le code suivant.

- Pouvez vous analyser les résultats ?
- Quelles sont vos conclusions sur l'influence de chacune des incertitudes ?
- Est-ce que vos conclusions sont en accord avec celles de l'analyse des corrélations ?

CODE DE LA QUESTION 5 A FAIRE TOURNER

In [8]:

```
Srank = err = []
for i in range(4):
    Srank.append(compSRank(Y,X,i))
err = []
for i in range(4):
    err.append(BootSRank(Y,X,i,100))
plt.bar(pred,Srank,  width = 0.5)
plt.errorbar(pred,Srank, yerr=err, fmt="o", color="r")
plt.show()
```



Utilisation d'un métamodèle

QUESTION 6 Nous allons construire un modèle de "machine learning" (métamodèle) pour remplacer le code de calcul qui nous a permis d'estimer les valeurs de *Y*. Définissez dans un premier temps une procédure de validation croisée **5** folds avec **1** répétition et un critère de performance "r^2". Compléter dans le code suivant les cases manquantes marqués par '##' et faites tourner le code.

In [9]:
```
kfold = model_selection.RepeatedKFold(n_splits=5,n_repeats=1, random_state=42)
scoring = 'r2'
```

QUESTION 7 A quoi sert l'argument random_state = 42 dans le code de la question 6 ?

QUESTION 8 Comment interpréter le critère de performance "r^2" ?

QUESTION 9 Nous allons comparer plusieurs modèles avec le code suivant. Quelles sont vos conclusions sur la performance des modèles ?

CODE DE LA QUESTION 9 A FAIRE TOURNER

In [10]:
```
models = {
    "Linear Regression": make_pipeline(StandardScaler(), LinearRegression()),
```

```
    "Support Vector Regressor (SVR)": make_pipeline(StandardScaler(), SVR(kernel
    "Random Forest": RandomForestRegressor(n_estimators=100, random_state=42, n_
    "Gradient Boosting": GradientBoostingRegressor(n_estimators=100,random_state
}
# CV
results = []
names = []
for name, model in models.items():
    cv_results = model_selection.cross_val_score(model, X, Y, cv=kfold, scoring=
    results.append(cv_results)
    names.append(name)
    msg = "%s: %f (%f)" % (name, cv_results.mean(), cv_results.std())
    print(msg)
```

```
Linear Regression: 0.565262 (0.085780)
Support Vector Regressor (SVR): -6.072152 (3.093301)
Random Forest: 0.393465 (0.126805)
Gradient Boosting: 0.328354 (0.198896)
```

QUESTION 10 Refaites la question 6 et 9 cette fois avec une validation croisée **3** folds et **10** répétitions. Vos conclusions ont-elles changé ?

In [11]:
```
kfold = model_selection.RepeatedKFold(n_splits=3,n_repeats=10, random_state=42)
scoring = 'r2'
models = {
    "Linear Regression": make_pipeline(StandardScaler(), LinearRegression()),
    "Support Vector Regressor (SVR)": make_pipeline(StandardScaler(), SVR(kernel
    "Random Forest": RandomForestRegressor(n_estimators=100, random_state=42, n_
    "Gradient Boosting": GradientBoostingRegressor(n_estimators=100,random_state
}
# CV
results = []
names = []
for name, model in models.items():
    cv_results = model_selection.cross_val_score(model, X, Y, cv=kfold, scoring=
    results.append(cv_results)
    names.append(name)
    msg = "%s: %f (%f)" % (name, cv_results.mean(), cv_results.std())
    print(msg)
```

```
Linear Regression: 0.560561 (0.080333)
Support Vector Regressor (SVR): -5.072637 (3.029164)
Random Forest: 0.440667 (0.110177)
Gradient Boosting: 0.361013 (0.179727)
```

QUESTION 11 Quelles sont vos recommendations finales:

- est-il raisonnable de faire l'analyse de sensibilité avec 300 réalisations aléatoires ?
- est-il pertinent d'utiliser les modèles de machine learning que vous avez testés ?
- quelles seraient la ou les prochaines étapes de cette étude ?